PayPulse

Complete repository files for GitHub

Create each file below with the exact name and folder shown. Do not commit a real **.env** file; only **.env.example** goes on GitHub. Replace YOUR-USERNAME in README.md with your GitHub username.

Folder layout

```
paypulse/
|-- public/
|   `-- index.html
|-- .env.example
|-- .gitignore
|-- LICENSE
|-- package.json
|-- README.md
`-- server.js
```

1. README.md

````
# PayPulse

Real-time PayPal payment tracking with AI-generated cash-flow insights, built for the PayPal
    hackathon.

Freelancers and small businesses often learn about payments, refunds, and disputes hours late.
    PayPulse receives PayPal webhooks the moment they happen, shows them on a live dashboard, and
    uses an AI model to turn recent activity into plain-language insights.

## How it uses PayPal (sandbox)
- **Webhooks:** `PAYMENT.CAPTURE.COMPLETED`, refund, and dispute events are received at
    `/webhook`, signature-verified through PayPal's `verify-webhook-signature` API, and de-
    duplicated by event ID.
- **Orders API (v2):** Creates and captures sandbox payments (`/api/create-order`, `/api/capture`)
     so payments can be generated live in the demo.
- **Sandbox accounts:** Business and personal test accounts simulate full payment flows with no
    real money.

## How it uses AI
- **Claude (Anthropic Messages API):** After each verified event, the last 20 events are sent to
    Claude, which returns short insights and flags anything unusual (refunds, disputes, gaps). The
     result is pushed to the dashboard in real time.

## Other tools and technologies
- Node.js 18+ and Express
- Server-Sent Events for live dashboard updates
- Plain HTML/CSS/JS frontend
- ngrok (local webhook tunnel for testing)
- Sponsor integrations: none

## Setup
1. **Clone and install**
   ```bash
   git clone https://github.com/YOUR-USERNAME/paypulse.git
   cd paypulse
   npm install
   ```
2. **Create a PayPal sandbox app** at https://developer.paypal.com > Apps & Credentials (Sandbox).
     Copy the Client ID and Secret.
3. **Configure environment**
   ```bash
   cp .env.example .env
   ```
   Fill in `PAYPAL_CLIENT_ID`, `PAYPAL_CLIENT_SECRET`, and `ANTHROPIC_API_KEY`.
4. **Start the app**
   ```bash
   npm start
   ```
5. **Expose it for webhooks**
   ```bash
   ngrok http 3000
   ```
   Set `BASE_URL` in `.env` to the ngrok https URL and restart.
6. **Register the webhook** in your sandbox app: URL `https://YOUR-NGROK-URL/webhook`, events
     `Payment capture completed`, `Payment capture refunded`, and the dispute events. Copy the
     **Webhook ID** into `PAYPAL_WEBHOOK_ID` in `.env` and restart.
7. **Try it:** open `http://localhost:3000`, click **Create $25 sandbox payment**, approve with a
    sandbox personal account, and watch the dashboard update.

## Project structure
```
paypulse/
|-- server.js        # Express server: webhooks, Orders API, AI insights, SSE
|-- public/index.html# Live dashboard
|-- .env.example     # Required environment variables (no secrets)
|-- package.json
|-- LICENSE          # MIT
````
````

```
`-- README.md
```

Limitations and next steps
- Events are stored in memory; use a database for production.
- Multi-currency support, cash-flow forecasting, and accounting integrations (e.g. QuickBooks).

License
MIT. See LICENSE.

2. package.json

```
{
  "name": "paypulse",
  "version": "1.0.0",
  "description": "Real-time PayPal payment tracking with AI-generated insights",
  "main": "server.js",
  "scripts": { "start": "node server.js", "dev": "node --watch server.js" },
  "engines": { "node": ">=18" },
  "license": "MIT",
  "dependencies": { "dotenv": "^16.4.5", "express": "^4.19.2" }
}
```

3. .env.example

```
# PayPal sandbox app credentials (developer.paypal.com > Apps & Credentials)
PAYPAL_CLIENT_ID=
PAYPAL_CLIENT_SECRET=
# Webhook ID shown after you register your webhook URL in the dashboard
PAYPAL_WEBHOOK_ID=
# Anthropic API key for AI insights
ANTHROPIC_API_KEY=
AI_MODEL=claude-sonnet-5-5
PORT=3000
# Public URL used for PayPal return links (your ngrok URL when testing)
BASE_URL=http://localhost:3000
```

4. .gitignore

```
node_modules/
.env
*.log
.DS_Store
```

5. LICENSE

```
MIT License

Copyright (c) 2026 PayPulse contributors

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

6. server.js

```
require('dotenv').config();
const express = require('express');
const path = require('path');

const {
  PAYPAL_CLIENT_ID, PAYPAL_CLIENT_SECRET, PAYPAL_WEBHOOK_ID, ANTHROPIC_API_KEY,
  PORT = 3000, AI_MODEL = 'claude-sonnet-5-5'
} = process.env;
const BASE_URL = process.env.BASE_URL || `http://localhost:${PORT}`;
const PP = 'https://api-m.sandbox.paypal.com'; // sandbox only

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

const events = [];        // in-memory store (swap for a DB in production)
const seen = new Set();   // idempotency: PayPal may retry/duplicate webhooks
const clients = new Set(); // SSE subscribers

// ---------- PayPal helpers ----------
async function ppToken() {
  const creds = `${PAYPAL_CLIENT_ID}:${PAYPAL_CLIENT_SECRET}`;
  const auth = Buffer.from(creds).toString('base64');
  const r = await fetch(`${PP}/v1/oauth2/token`, {
    method: 'POST',
    headers: {
      Authorization: `Basic ${auth}`,
      'Content-Type': 'application/x-www-form-urlencoded'
    },
    body: 'grant_type=client_credentials'
  });
  if (!r.ok) throw new Error(`PayPal auth failed: ${r.status}`);
  return (await r.json()).access_token;
}

async function verifyWebhook(req) {
  const h = req.headers;
  const r = await fetch(`${PP}/v1/notifications/verify-webhook-signature`, {
    method: 'POST',
    headers: { Authorization: `Bearer ${await ppToken()}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({
      auth_algo: h['paypal-auth-algo'],
      cert_url: h['paypal-cert-url'],
      transmission_id: h['paypal-transmission-id'],
      transmission_sig: h['paypal-transmission-sig'],
      transmission_time: h['paypal-transmission-time'],
      webhook_id: PAYPAL_WEBHOOK_ID,
      webhook_event: req.body
    })
  });
  const data = await r.json();
  return data.verification_status === 'SUCCESS';
}

// ---------- AI insights (Anthropic API) ----------
async function getInsight() {
  if (!ANTHROPIC_API_KEY || events.length === 0) return null;
  const summary = events.slice(0, 20).map(e =>
    `${e.time} | ${e.type} | ${e.amount ?? '-'} ${e.currency ?? ''}`).join('\n');
  try {
    const r = await fetch('https://api.anthropic.com/v1/messages', {
      method: 'POST',
      headers: {
        'x-api-key': ANTHROPIC_API_KEY,
        'anthropic-version': '2023-06-01',
        'content-type': 'application/json'
      },
```

```
      body: JSON.stringify({
        model: AI_MODEL,
        max_tokens: 300,
        system:
          'You are a cash-flow assistant for freelancers. ' +
          'Given recent PayPal events, reply with 2-3 short plain-language insights ' +
          'and flag anything unusual (refunds, disputes, gaps). No preamble.',
        messages: [{ role: 'user', content: summary }]
      })
    });
    const data = await r.json();
    return data.content?.map(c => c.text || '').join('') || null;
  } catch (e) { console.error('AI error', e.message); return null; }
}

function broadcast(payload) {
  const msg = `data: ${JSON.stringify(payload)}\n\n`;
  clients.forEach(c => c.write(msg));
}

// ---------- Routes ----------
app.post('/webhook', async (req, res) => {
  try {
    if (!(await verifyWebhook(req))) return res.sendStatus(400);
  } catch (e) { console.error(e.message); return res.sendStatus(500); }

  const ev = req.body;
  if (seen.has(ev.id)) return res.sendStatus(200); // duplicate
  seen.add(ev.id);

  const res_ = ev.resource || {};
  const record = {
    id: ev.id,
    type: ev.event_type,
    time: ev.create_time,
    amount: res_.amount?.value ?? res_.gross_amount?.value ?? null,
    currency: res_.amount?.currency_code ?? res_.gross_amount?.currency_code ?? null
  };
  events.unshift(record);
  res.sendStatus(200);

  broadcast({ event: record });
  const insight = await getInsight();
  if (insight) broadcast({ insight });
});

app.get('/api/events', (_req, res) => res.json(events));

app.get('/api/stream', (req, res) => {
  res.set({
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive'
  });
  res.flushHeaders();
  clients.add(res);
  req.on('close', () => clients.delete(res));
});

app.post('/api/insight', async (_req, res) => {
  res.json({ insight: await getInsight() });
});

// Create a sandbox order (Orders API); open the returned link with a sandbox buyer
app.post('/api/create-order', async (req, res) => {
  try {
    const value = String(req.body?.amount || '25.00');
    const r = await fetch(`${PP}/v2/checkout/orders`, {
      method: 'POST',
      headers: { Authorization: `Bearer ${await ppToken()}`, 'Content-Type': 'application/json' },
```

```
      body: JSON.stringify({
        intent: 'CAPTURE',
        purchase_units: [{ amount: { currency_code: 'USD', value } }],
        payment_source: {
          paypal: {
            experience_context: {
              return_url: `${BASE_URL}/api/capture`,
              cancel_url: BASE_URL,
              user_action: 'PAY_NOW'
            }
          }
        }
      })
    });
    const order = await r.json();
    const approve = order.links
      ?.find(l => ['approve', 'payer-action'].includes(l.rel))?.href;
    res.json({ id: order.id, approve });
  } catch (e) { res.status(500).json({ error: e.message }); }
});

// PayPal redirects here after the buyer approves; we capture the payment
app.get('/api/capture', async (req, res) => {
  try {
    await fetch(`${PP}/v2/checkout/orders/${req.query.token}/capture`, {
      method: 'POST',
      headers: { Authorization: `Bearer ${await ppToken()}`, 'Content-Type': 'application/json' }
    });
  } catch (e) { console.error(e.message); }
  res.redirect('/'); // the PAYMENT.CAPTURE.COMPLETED webhook updates the dashboard
});

app.listen(PORT, () => console.log(`PayPulse running at ${BASE_URL}`));
```

7. public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>PayPulse</title>
<style>
  body {
    font-family: system-ui, sans-serif;
    max-width: 760px;
    margin: 2rem auto;
    padding: 0 1rem;
    background: #f6f8fb;
    color: #1b2430;
  }
  h1 { margin-bottom: 0; }
  .sub { color: #5b6777; margin-top: .2rem; }
  .card {
    background: #fff;
    border-radius: 12px;
    padding: 1rem 1.2rem;
    margin: 1rem 0;
    box-shadow: 0 1px 4px #0001;
  }
  button {
    background: #0070ba;
    color: #fff;
    border: 0;
    border-radius: 8px;
    padding: .6rem 1rem;
    cursor: pointer;
    font-size: 1rem;
  }
  table { width: 100%; border-collapse: collapse; }
  td, th { text-align: left; padding: .45rem; border-bottom: 1px solid #eee; }
  .dot {
    display: inline-block;
    width: 10px;
    height: 10px;
    border-radius: 50%;
    background: #2ecc71;
    margin-right: 6px;
    animation: p 1.5s infinite;
  }
  @keyframes p { 50% { opacity: .3; } }
</style>
</head>
<body>
<h1><span class="dot"></span>PayPulse</h1>
<p class="sub">Live PayPal activity with AI insights (sandbox)</p>

<div class="card">
  <button id="pay">Create $25 sandbox payment</button>
  <span id="msg"></span>
</div>

<div class="card">
  <h3>AI insight</h3>
  <p id="insight">Waiting for payments...</p>
</div>
<div class="card">
  <h3>Live events</h3>
  <table>
    <thead><tr><th>Time</th><th>Type</th><th>Amount</th></tr></thead>
    <tbody id="rows"></tbody>
  </table>
</div>
```

```
<script>
const rows = document.getElementById('rows');

function add(e, top = true) {
  const tr = document.createElement('tr');
  const amount = e.amount ? `${e.amount} ${e.currency}` : '-';
  [new Date(e.time).toLocaleTimeString(), e.type, amount].forEach(t => {
    const td = document.createElement('td');
    td.textContent = t;
    tr.appendChild(td);
  });
  top ? rows.prepend(tr) : rows.appendChild(tr);
}

fetch('/api/events')
  .then(r => r.json())
  .then(list => list.forEach(e => add(e, false)));

const es = new EventSource('/api/stream');
es.onmessage = m => {
  const d = JSON.parse(m.data);
  if (d.event) add(d.event);
  if (d.insight) document.getElementById('insight').textContent = d.insight;
};

document.getElementById('pay').onclick = async () => {
  const r = await fetch('/api/create-order', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ amount: '25.00' })
  });
  const { approve } = await r.json();
  if (approve) window.open(approve, '_blank');
  else document.getElementById('msg').textContent =
    ' Could not create order. Check your .env.';
};
</script>
</body>
</html>
```